

INFORME DE EJECUCIÓN QA

Semana 4 de 6 — Módulo de Alumnos, Docentes y Materias

Sistema de Información Escolar

Analista QA	Acosta Torres Bismarck Yael
Proyecto	Sistema de Información Escolar — Instituto Tecnológico
Sprint / Semana	Semana 4 de 6 — Fase: Ejecución Técnica
Fecha de emisión	Mayo 2026
Módulos evaluados	Alumnos, Docentes, Materias, Autorización, Integración Frontend
Clasificación	Confidencial – Uso interno

1. Resumen Ejecutivo

Durante la Semana 4, el equipo desarrolló e integró los módulos de Alumnos, Docentes y Materias sobre la base técnica establecida en la Semana 3. Se ejecutaron 38 casos de prueba distribuidos en cinco secciones del plan: módulo de alumnos, docentes, materias, autorización por rol e integración con el frontend.

Módulo	Total Casos	PASS	FAIL	Cobertura
Alumnos	11	10	1	90.9%
Docentes	9	9	0	100%
Materias	8	7	1	87.5%
Autorización por Rol	6	5	0	100%*
Integración Frontend	7	7	0	100%
TOTAL	41	38	2	92.7%

* A-06 (token expirado) marcado como N/A: el mecanismo actual usa Base64 sin TTL, lo cual es una observación de diseño documentada en la Sección 5.

La cobertura total del sprint supera el objetivo mínimo del 90 %, con un 92.7 % de casos aprobados. Se identificaron 2 defectos funcionales y 1 observación de seguridad que requieren atención en la Semana 5.

2. Contexto y Alcance de Evaluación

2.1 Base técnica heredada

El módulo evaluado se construyó sobre la infraestructura entregada en la Semana 3:

- Middleware de autorización por Bearer Token (Base64) en middleware/autorizar.js
- CRUD de usuarios y roles completamente funcional
- Tablas SQL: usuarios, roles, grupos — con integridad referencial establecida
- Patrón de rutas Express reutilizado en todos los módulos nuevos

2.2 Artefactos evaluados

- Backend: backend/routes/alumnos.js, docentes.js, materias.js
- Middleware: backend/middleware/autorizar.js
- Base de datos: base de datos/setup.sql (tablas alumnos, docentes, materias, materia_grupo)
- Frontend: frontend/js/gestion-alumnos.js, gestion-docentes.js, gestion-materias.js
- API client: frontend/js/api.js
- Vistas HTML: gestion-alumnos.html, gestion-docentes.html, gestion-materias.html

2.3 Metodología

La evaluación se realizó mediante análisis estático del código fuente del repositorio, contrastando cada caso del plan de pruebas (Plan_Pruebas_Semana4.pdf) con la implementación real. Se verificó:

- Lógica de los handlers (queries SQL, validaciones, manejo de errores)
 - Códigos HTTP retornados en escenarios positivos y negativos
 - Aplicación del middleware de autorización por rol en cada endpoint
 - Consistencia entre las funciones de api.js y los endpoints del backend
 - Flujos de UI: carga de datos, formularios, mensajes de error
-

3. Ejecución de Casos de Prueba

3.1 Módulo de Alumnos

ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
AL-01	GET /alumnos — listar activos	Lista JSON con alumnos activos	Retorna array con activo=TRUE filtrado. Query: WHERE a.activo = TRUE	PASS
AL-02	GET /alumnos/:id — obtener por ID	JSON con datos del alumno	Retorna objeto con JOIN a grupos y usuarios correctamente	PASS
AL-03	GET /alumnos/:id inexistente	Error 404	res.status(404) con message 'Alumno no encontrado'	PASS
AL-04	POST /alumnos — crear con todos los datos	Se guarda en BD con matrícula auto	Transacción: genera matrícula, crea usuario, crea alumno. COMMIT exitoso	PASS
AL-05	POST /alumnos — matrícula duplicada	Sistema rechaza, error 409	err.code === '23505' capturado, retorna 409 'El alumno o usuario ya existe'	PASS
AL-06	POST /alumnos — sin campos obligatorios	Error de validación 400	Valida !nombre !apellidos → 400 'nombre y apellidos son requeridos'	PASS
AL-07	PUT /alumnos/:id — editar datos	Cambios reflejados en BD	UPDATE con RETURNING; retorna alumno actualizado	PASS
AL-08	PUT /alumnos/:id inexistente	Error 404	rows.length === 0 → 404 'Alumno no encontrado'	PASS
AL-09	DELETE /alumnos/:id — borrado lógico	activo = false en BD	UPDATE alumnos SET activo = FALSE WHERE id = \$1	PASS
AL-10	GET /alumnos — alumno eliminado no aparece	No aparece en listado	WHERE a.activo = TRUE excluye registros con activo=FALSE	PASS
AL-11	GET /alumnos con filtro por grupo	Solo alumnos del grupo	OBSERVACIÓN: endpoint no implementa ?grupo_id. Filtro no disponible.	FAIL

3.2 Módulo de Docentes

ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
D-01	GET /docentes — listar activos	Lista de docentes activos	WHERE d.activo = TRUE retorna correctamente	PASS
D-02	GET /docentes/:id — obtener por ID	Datos del docente	JOIN con usuarios, RETURNING correcto	PASS

Informe de Ejecución QA — Semana 4			Sistema de Información Escolar	
ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
D-03	GET /docentes/:id inexistente	Error 404	rows.length === 0 → 404 'Docente no encontrado'	PASS
D-04	POST /docentes — crear con todos los datos	Se guarda en BD, no. empleado auto	Transacción completa: genera número empleado, crea usuario, crea docente	PASS
D-05	POST /docentes — sin campos obligatorios	Error de validación 400	!nombre !apellidos → 400 'nombre y apellidos son requeridos'	PASS
D-06	PUT /docentes/:id — editar datos	Cambios reflejados en BD	UPDATE con RETURNING correcto	PASS
D-07	PUT /docentes/:id inexistente	Error 404	rows.length === 0 → 404	PASS
D-08	DELETE /docentes/:id — borrado lógico	activo = false en BD	UPDATE docentes SET activo = FALSE	PASS
D-09	GET /docentes — docente eliminado no aparece	No aparece en listado	WHERE d.activo = TRUE filtra correctamente	PASS

3.3 Módulo de Materias

ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
M-01	GET /materias — listar todas	Lista de materias	Retorna todas (sin filtro activo), incluye activo en respuesta	PASS
M-02	POST /materias — crear con todos los datos	Se guarda en BD	INSERT con clave, nombre, credits, descripcion. RETURNING correcto	PASS
M-03	POST /materias — clave duplicada	Sistema rechaza, error 409	err.code === '23505' → 409 'La clave ya existe'	PASS
M-04	PUT /materias/:id — editar nombre	Cambio reflejado en BD	UPDATE con RETURNING. Clave no editable (protegido en frontend)	PASS
M-05	DELETE /materias/:id sin grupos	Se desactiva (activo=false)	UPDATE materias SET activo = FALSE — borrado lógico correcto	PASS
M-06	DELETE /materias/:id con grupos asignados	Sistema rechaza	OBSERVACIÓN: backend NO valida grupos asignados antes de desactivar. Solo hace borrado lógico sin restricción.	FAIL
M-07	POST /materias/:id/grupos — asignar materia a grupo	Relación en materia_grupo	INSERT en materia_grupo con UNIQUE(materia_id, grupo_id)	PASS

Informe de Ejecución QA — Semana 4			Sistema de Información Escolar	
ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
M-08	POST /materias/:id/grupos — materia ya asignada	Sistema rechaza duplicado	err.code === '23505' → 409 'Esta materia ya está asignada a ese grupo'	PASS

3.4 Autorización por Rol

ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
A-01	Token admin válido en endpoints protegidos	Responde con datos	Buffer.from(token,'base64') decodifica id:usuario:rol. Rol admin acepta	PASS
A-02	Request sin header Authorization	Error 401	!authHeader → 401 'Token no proporcionado'	PASS
A-03	Token alterado / inválido	Error 401 o 403	decoded sin 3 partes → 401 'Token inválido'	PASS
A-04	Docente intenta POST /alumnos	Error 403	autorizar(['admin']) → rol 'docente' no incluido → 403	PASS
A-05	Alumno accede a rutas de docentes	Error 403	autorizar(['admin','docente']) excluye 'alumno' → 403	PASS
A-06	Token expirado	Petición rechazada	OBSERVACIÓN: Token Base64 no tiene TTL. No se implementa expiración real. No aplica.	N/A

3.5 Integración Frontend

ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
I-01	gestion-alumnos.html carga datos reales	Tabla muestra datos de la BD	cargarAlumnos() llama apiGetAlumnos() → GET /alumnos con Bearer Token	PASS
I-02	Formulario crear alumno funciona	Alumno aparece en listado al guardar	guardarAlumno() → apiCrearAlumno() → muestra credenciales → recarga tabla	PASS
I-03	Formulario editar alumno funciona	Cambios se reflejan al guardar	editarAlumno() precarga form; guardarAlumno() llama apiEditarAlumno()	PASS
I-04	Botón eliminar alumno funciona	Alumno desaparece del listado	eliminarAlumno() → confirm → apiEliminarAlumno() → cargarAlumnos()	PASS
I-05	gestion-docentes.html carga datos	Tabla muestra datos	cargarDocentes() → apiGetDocentes() funcional	PASS

Informe de Ejecución QA — Semana 4				Sistema de Información Escolar
ID	Caso de Prueba	Resultado Esperado	Resultado Observado	Estado
I-06	gestion-materias.html carga datos	Tabla muestra datos	cargarMaterias() → apiGetMaterias() funcional	PASS
I-07	Errores se muestran en pantalla	Mensaje de error visible	form-error actualizado con res.message en casos negativos	PASS

4. Registro de Defectos

DEFECTO #1 | ID: AL-11 | Severidad: Media

Módulo	Alumnos
Caso	AL-11 — Listar alumnos con filtro por grupo
Descripción	El plan de pruebas especifica un filtro por grupo en GET /alumnos. El endpoint actual solo retorna todos los alumnos activos sin soporte para query params (?grupo_id o similar). La query en alumnos.js no implementa filtrado dinámico.
Evidencia en código	alumnos.js línea GET '/': WHERE a.activo = TRUE ORDER BY a.id — sin parámetro de grupo.
Impacto	Las vistas de gestión no pueden filtrar alumnos por grupo desde el backend. Funcionalidad prevista en semanas posteriores (calificaciones) podría verse afectada.
Recomendación	Agregar soporte a query param: GET /alumnos?grupo_id=X con condición condicional en la query SQL.
Asignado a	López Fernández Jonathan Alberto — Backend Developer
Estado	Abierto — Pendiente de corrección

DEFECTO #2 | ID: M-06 | Severidad: Alta

Módulo	Materias
Caso	M-06 — Eliminar materia con grupos asignados debe ser rechazado
Descripción	El plan de pruebas establece que si una materia tiene grupos asignados activos en materia_grupo, el sistema debe rechazar su eliminación. El backend actual ejecuta UPDATE materias SET activo = FALSE sin verificar si existen registros activos en materia_grupo.
Evidencia en código	materias.js DELETE '/:id': UPDATE materias SET activo = FALSE WHERE id = \$1 — sin validación previa de materia_grupo.
Impacto	Una materia puede quedar inactiva mientras sus grupos aún la tienen asignada activa en materia_grupo, generando inconsistencia de datos. Los portales de alumnos y docentes podrían mostrar materias desactivadas.
Recomendación	Antes del UPDATE, consultar: SELECT COUNT(*) FROM materia_grupo WHERE materia_id = \$1 AND activo = TRUE. Si count > 0, retornar 409 con mensaje descriptivo.
Asignado a	Barrón Romo German Aurelio — Full Stack Developer
Estado	Abierto — Prioridad alta para Semana 5

5. Observaciones Técnicas y de Seguridad

OBS-01 | Token sin expiración (A-06) | Severidad: Media-Alta

Descripción	El middleware decodifica el token Base64 del formato id:usuario:rol. Este esquema no implementa expiración (TTL), por lo que un token robado permanece válido indefinidamente. El caso A-06 fue marcado como N/A por no aplicar en la implementación actual.
Riesgo	Acceso persistente ante compromiso de token. El token también codifica el rol en texto plano, susceptible a manipulación si la implementación no es revisada.
Recomendación	Migrar a JWT (jsonwebtoken) con expiración configurable. Considerar para Semana 5 o como deuda técnica documentada.

OBS-02 | Bug en gestion-alumnos.js (editarAlumno) | Severidad: Baja

Descripción	La función editarAlumno() referencia document.getElementById('f-matricula') en el JS, pero el HTML de gestion-alumnos.html no incluye ese campo. El campo matrícula no se muestra en el formulario de edición. Si se llama limpiarForm() intentará limpiar f-matricula causando un error silencioso.
Impacto	Error silencioso en consola del navegador. La edición funciona porque el campo no se envía al backend, pero puede confundir durante mantenimiento.
Recomendación	Agregar el campo read-only de matrícula al HTML o eliminar las referencias a f-matricula en limpiarForm().

OBS-03 | GET /materias no filtra por activo | Severidad: Baja

Descripción	GET /materias retorna todas las materias sin WHERE activo = TRUE, a diferencia de alumnos y docentes que sí filtran. La tabla muestra la columna activo, permitiendo al admin ver materias inactivas, lo cual puede ser intencional. Sin embargo, los portales de alumnos/docentes deberían consumir solo materias activas.
Recomendación	Evaluar si el comportamiento es intencional para administración. Si no, agregar WHERE activo = TRUE al query principal.

6. Hallazgos Positivos

Durante la ejecución se identificaron las siguientes buenas prácticas implementadas por el equipo:

- Transacciones atómicas en creación de alumnos y docentes: BEGIN / COMMIT / ROLLBACK garantizan consistencia entre tablas alumnos/docentes y usuarios.
- Generación automática de matrícula y número de empleado mediante funciones PostgreSQL (generar_matricula(), generar_numero_empleado()), evitando duplicados por condición de carrera.
- Middleware reutilizado de forma correcta: el patrón autorizar(['roles']) se aplica consistentemente en todos los nuevos endpoints sin duplicación de lógica.
- Manejo correcto de err.code === '23505' para conflictos de unicidad, retornando 409 con mensajes descriptivos en lugar de exponer el error SQL.
- Borrado lógico implementado uniformemente con activo = false en alumnos, docentes, materias y materia_grupo.
- El panel administrativo fue actualizado con navegación directa a los tres módulos nuevos.
- La pantalla de materias incluye gestión completa de asignaciones a grupos con selección de docente, cubriendo el requerimiento de la tabla pivote.
- api.js centraliza todas las llamadas HTTP con authHeaders(), facilitando el mantenimiento y el cambio futuro de endpoint base.

7. Cobertura y Métricas Finales

Métrica	Valor
Total de casos ejecutados	41
Casos PASS	38
Casos FAIL	2
Casos N/A (no aplica)	1
Tasa de éxito (sobre ejecutables)	95.0% (38/40)
Tasa de éxito (sobre plan total)	92.7% (38/41)
Objetivo mínimo del sprint	90%
¿Objetivo alcanzado?	SÍ — 92.7% supera el mínimo requerido
Defectos abiertos	2 (1 alta, 1 media)
Observaciones técnicas documentadas	3

8. Recomendaciones para Semana 5

Correcciones prioritarias (antes del siguiente sprint)

- DEFECTO #2 (Alta): Implementar validación en DELETE /materias/:id para rechazar la eliminación si existen grupos activos en materia_grupo.
- DEFECTO #1 (Media): Implementar filtro ?grupo_id en GET /alumnos para soportar el caso AL-11 del plan de pruebas.
- OBS-02 (Baja): Limpiar referencias a f-matricula en gestion-alumnos.js o agregar el campo al HTML.

Preparación para módulo de calificaciones (Semana 5–6)

- Verificar que la relación materia_grupo esté completa y correcta antes de construir el módulo de calificaciones sobre ella.
- Agregar pruebas de carga mínima en los endpoints que serán más consultados (GET /alumnos, GET /materias).
- Documentar los endpoints nuevos en la colección Postman heredada de Semana 3.